database management has to cater to data from multiple external data sources such as customers, GPS, mobile devices, the general public, point-of-sale devices, sensor data and so on. There are new kinds of data, such as Web pages, digitised content such as books and records, music, videos, photos, satellite images, scientific data, messages, tweets and sensor data—each with different data-processing requirements. Traditionally, databases only needed to cater to enterprise-centric workloads such as OLTP/OLAP. However, Big Data has ushered in a whole new set of data-centric workloads, such as Web search, massively multi-player online games, online message systems like Twitter, sensor networks, social network analysis, media streaming, photo processing, etc. The data management needs of all these data-types cannot be met by traditional database architectures. These data-centric workloads have different characteristics in the following areas:

a) Response time requirements—such as real-time versus non-real-time.
b) Data types:
   - Structured data that fits in well with traditional RDBMS schemas.
   - Semi-structured data, like XML or email.
   - Fully unstructured data, such as binary or sensor data.
c) Processing complexity:
   - Simple data operations, such as aggregate, sort or upload/download, with a low compute-to-data-access ratio.
   - Medium compute complexity operations on data, such as pattern matching, search or encryption.
   - Complex processing, such as video encoding/decoding, analytics, prediction, etc.

Big Data has brought forth the issue of 'database as the bottleneck' for many of these data-centric workloads, due to their widely varying requirements. The inability of the traditional RDBMS to scale up to massive data sets led to alternatives such as Data Sharding and Scale-Out Architectures, and subsequently to the NoSQL movement, which we will discuss later.

Database evolution from the 1960s onwards

There has been a huge evolution from the simple systems of the 1960s to what we have today. Let's look at some of the stages in this evolution.

Flat and hierarchical databases

In a flat model, the data is stored as records and delimiters in a simple file. In hierarchical data, model data is organised into a tree-like structure using parent/child relationships with a one-to-many ratio (see *http://en.wikipedia.org/wiki/Tree_data_structure*). This was the pre-cursor to relational databases, with no support for querying, and the responsibility of data base administration was ad-hoc, being left to the individual maintainer to take care of, without any software support.

Relational databases

A relational database is a set of relations such that the data satisfies the predicates which describe the constraints on the possible values and their combinations. It provides a declarative method for specifying data and queries. RDBMS software describes data structures for storing the data, as well as the retrieval procedures. Relations are represented as tables in the database. A table describes a specific entity type, and all attributes of a specific record are listed under an entity type. Each individual record is represented as a row, and an attribute as a column. This is the relational database model as proposed by Codd in the 1960s.

The relational model was the first database model to be described in the formal terms of relational algebra. The relational database model went on to become the de-facto standard for all enterprise database management systems from the 1960s till the late nineties.

Object-oriented databases

In the mid-eighties, object-oriented databases were proposed, in order to allow greater programming flexibility by allowing objects to be directly stored in databases. Relations in a relational database represent behaviours, whereas interconnection between objects cannot be represented easily in the relational form. OODBMS were intended to address this shortcoming. In an OODBMS, application data is represented by persistent objects that match the objects used in the programming language. However, object-oriented databases were not very successful, since they were more focused on addressing the programmer needs rather than the business intelligence needs of the organisation.

Columnar databases

These organise data from the same attribute as columns of values, as opposed to storing it as rows on disk. This results in large I/O savings in analytical and data-warehousing type of data retrieval, that largely accesses a set of columns. Note that columnar databases are relational, and support ACID semantics, as well as providing SQL support. A popular columnar database that offers state-of-the-art analytical capabilities is Vertica. It is based on CStore, a column-oriented academic database research project described in the paper *http://db.csail.mit.edu/projects/cstore/vldb.pdf*. Vertica has decoupled its Read Store (optimised for read-only accesses) from the Write Store (optimised for high performance updates and inserts) to evolve a hybrid model that offers excellent scalability. The high degree of compression that can be achieved due to the nature of columnar data, grouped together with the rest of the